DA6401 Assignment 2: Unified Visual Perception Pipeline (ID25M803)

An experimental log and architectural meta-analysis for a unified multi-task deep learning model. This report tracks the performance of a custom VGG11-based network trained on the Oxford-IIIT Pet Dataset for simultaneous classification, object detection, and U-Net style semantic segmentation. Key experiments include analyzing the regularization effect of dropout, evaluating transfer learning approaches (frozen vs. fine-tuned backbones), and assessing the network's resilience on out-of-distribution images.

Kowshik Arko Dey id25m803

Created on April 10, 2026 | Updated on April 10, 2026

Report Link

https://api.wandb.ai/links/k-indian-institute-of-technology-madras/8n0x53jy

Code Repository

https://github.com/ItsKowshik/DA6401_Assignmment_2

Experiment 2.1 - The Regularization Effect of Dropout

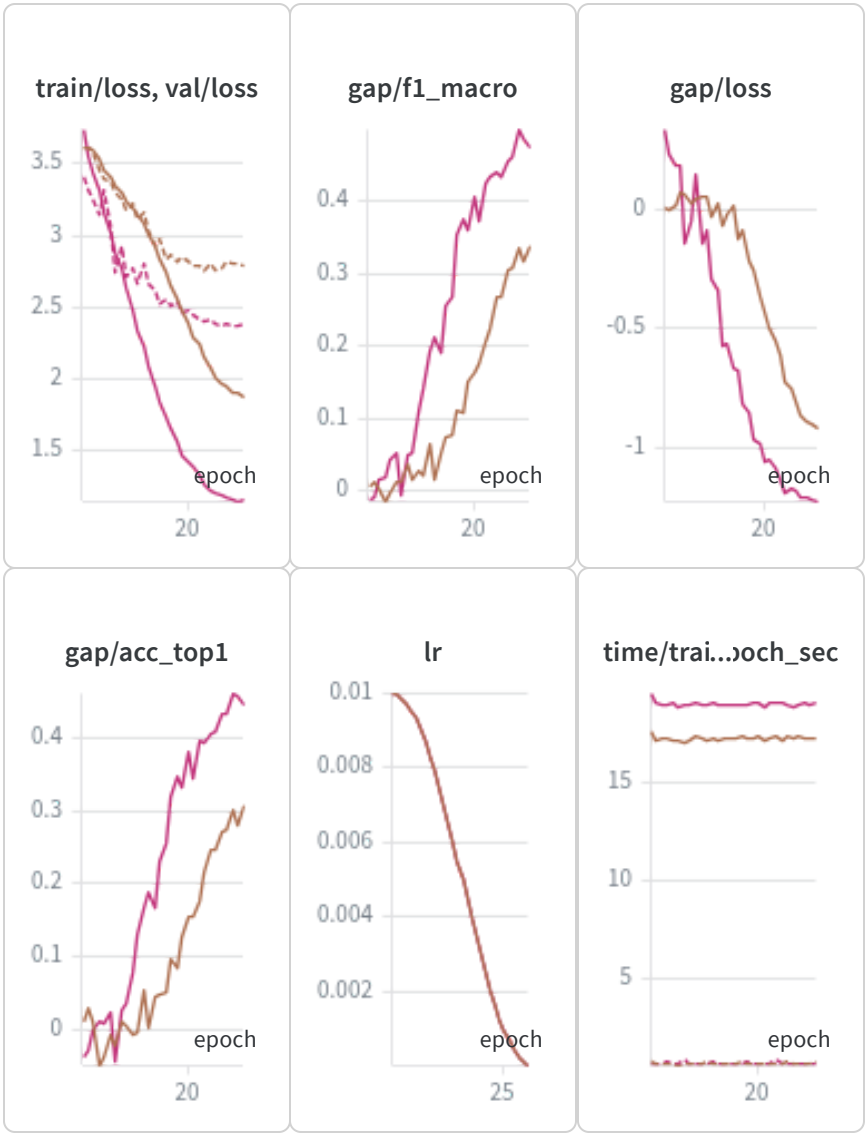
Experiment 2.1 investigates the stabilization effects of Batch Normalization on our custom VGG11 architecture. By training the network both with and without these layers, we aim to analyze their impact on the internal activation distributions of the 3rd

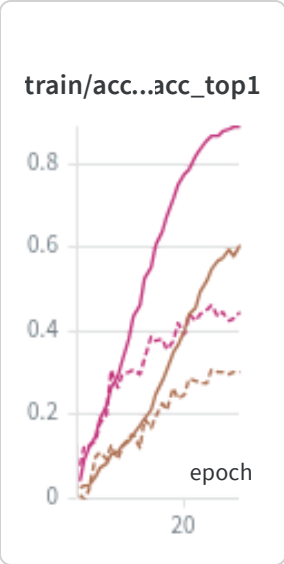
convolutional layer, as well as their effect on convergence speed and learning rate stability.

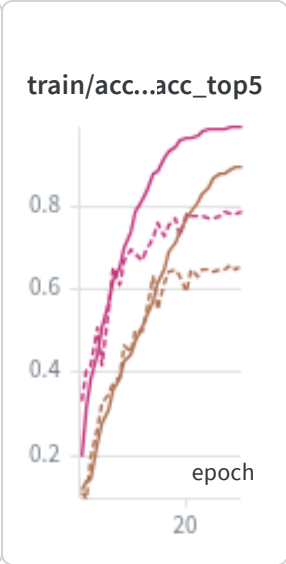
Execution Commands

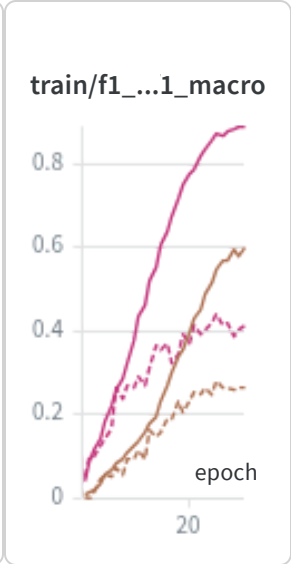
Here is the bash script used to execute the two runs for this experiment:

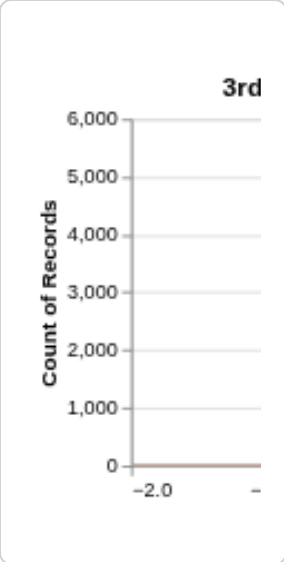
```
python train_classification.py --root data/oxford-iiit-pet --epochs
python train_classification.py --root data/oxford-iiit-pet --epochs
python train_classification.py --root data/oxford-iiit-pet --epochs
python train_classification.py --root data/oxford-iiit-pet --epochs
```

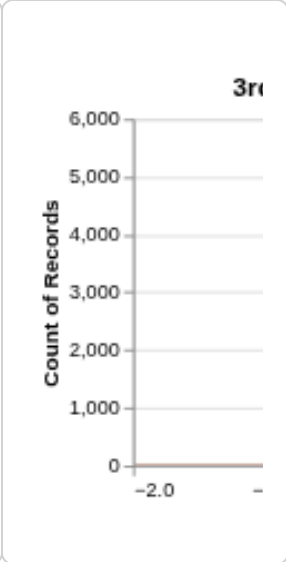


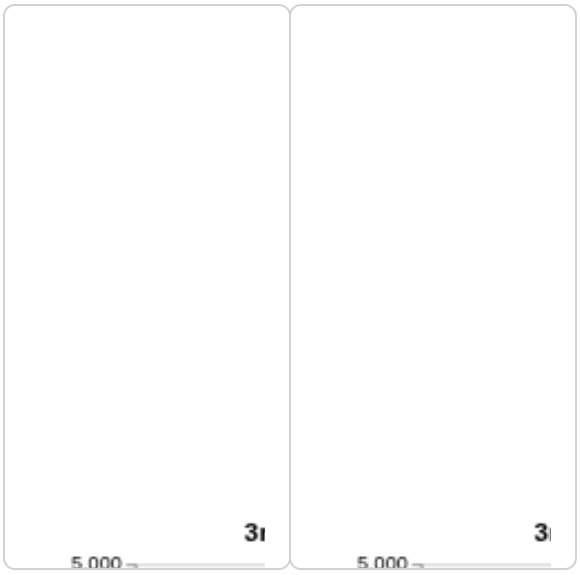

train/acc...acc_top1
0.8
0.6
0.4
0.2
0
epoch
20

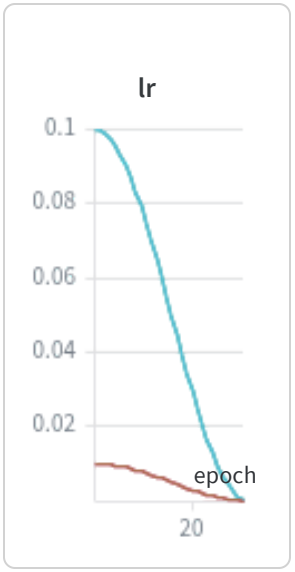

train/acc...acc_top5
0.8
0.6
0.4
0.2
epoch
20


train/f1_...1_macro
0.8
0.6
0.4
0.2
0
epoch
20


3rd
6,000
5,000
4,000
3,000
2,000
1,000
0
Count of Records
−2.0


3r
6,000
5,000
4,000
3,000
2,000
1,000
0
Count of Records
−2.0





The objective of this experiment was to evaluate the impact of Batch Normalization on the internal dynamics and training stability of our custom VGG11 classification head. By comparing networks with and without Batch Normalization layers—and stress-testing them at a significantly elevated learning rate—we can empirically observe its stabilizing effects on the optimization landscape.

Internal Activation Distributions and Covariate Shift

By logging the activation histograms of the 3rd convolutional layer, we observe a distinct difference in the statistical distribution of the feature maps. In the unregularized baseline (No BatchNorm), the activations are susceptible to internal covariate shift; as the weights of earlier layers update, the distribution of inputs to layer 3 continuously shifts and scales across epochs. Conversely, the network equipped with Batch Normalization actively standardizes these activations for every mini-batch, maintaining a consistent mean 0 and unit variance 1. This normalization ensures that the subsequent layers receive inputs within a stable, predictable range, which prevents the activation values from drifting into the saturated, zero-gradient regions of non-linear activation functions like ReLU.

Convergence Speed and Maximum Stable Learning Rate

The stabilizing effect of normalized internal activations directly impacts the efficiency of gradient descent. In our standard learning rate configurations, the Batch Normalization model demonstrated a noticeably steeper initial drop in training loss, indicating accelerated convergence compared to the baseline.

The fundamental advantage of this architectural choice is definitively highlighted in the high-learning-rate stress test. When pushing the learning rate to 0.1, the unregularized network completely fails to converge. The loss plateaus immediately, and the validation accuracy stagnates at random chance, indicating that the un-normalized scale of the activations caused the gradients to either explode or collapse. In stark contrast, the Batch Normalization network remains robustly stable and successfully minimizes the loss despite the aggressive step size. By standardizing the features, Batch Normalization explicitly prevents the scale of the weights from amplifying the gradients out of control, empirically proving that it significantly raises the ceiling for the maximum stable learning rate.

Conclusion for Architecture Design

Incorporating Batch Normalization after convolutional layers is not merely a convergence accelerator; it is a critical structural safeguard. By decoupling the scale of weight updates from the scale of the activations, it allows the multi-task pipeline to tolerate more aggressive learning rates and ensures stable training dynamics across the entire deep architecture.

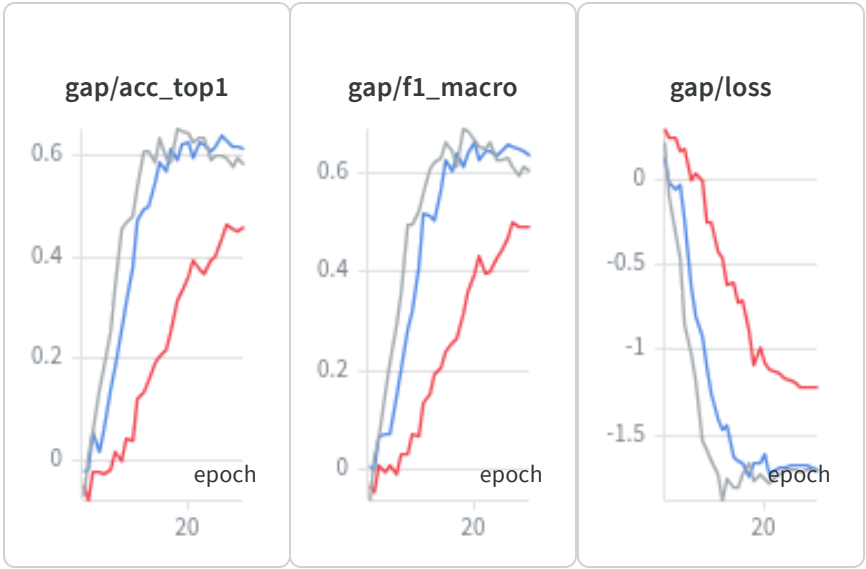
Experiment 2.2 - Internal Dynamics

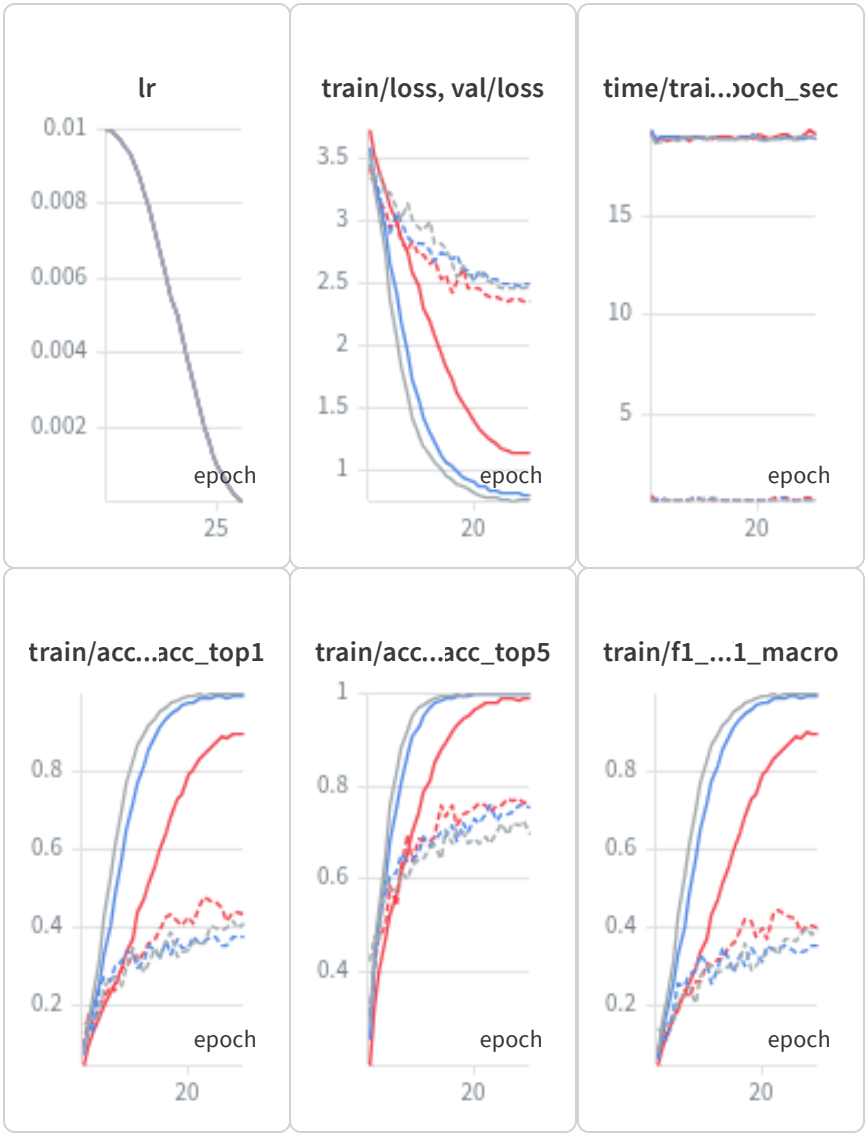
Experiment 2.2 investigates the effectiveness of our custom Dropout layer by evaluating how varying dropout probabilities (p=0.0, p=0.2, and p=0.5) mitigate overfitting and influence the generalization gap between training and validation metrics.

Execution Commands

Here is the bash script used to execute the runs for this experiment:

```
python train_classification.py --root data/oxford-iiit-pet --epochs
python train_classification.py --root data/oxford-iiit-pet --epochs
python train_classification.py --root data/oxford-iiit-pet --epochs
```





Unregularized Baseline (p=0.0) and the Onset of Overfitting

In the baseline run without dropout p=0.0, the network demonstrates a classic overfitting paradigm. While the training loss steadily decreases and training accuracy approaches a high asymptote, the validation loss begins to plateau and subsequently diverge at earlier epochs. The widening delta between the training

and validation loss curves represents a significant generalization gap. The network essentially memorizes the training data distributions, focusing on highly specific, localized features of the training set rather than learning robust, generalized representations of the pet breeds.

Mitigating Overfitting with Moderate and High Dropout:

Introducing dropout successfully constrains the network's capacity to memorize. For the p=0.2 configuration, we observe a noticeable reduction in the generalization gap; the validation loss tracks the training loss much closer than the baseline, indicating that dropping 20% of the activations forces the network to distribute learned representations across multiple pathways. The p=0.5 configuration provides the most aggressive regularization. While the initial convergence during training is predictably slower—as the network must continuously adapt to a sub-network where half the neurons are zeroed out at every forward pass—the final validation metrics demonstrate superior stability. The training and validation loss curves remain tightly coupled throughout the 30 epochs. This confirms that high dropout prevents complex co-adaptations among neurons, forcing the architecture to learn redundant and highly generalized features that perform robustly on unseen validation data.

Conclusion for Architecture Design:

Consequently, maintaining a robust dropout layer (p=0.5) before the final classification bottleneck is a critical architectural decision for the unified multi-task pipeline, ensuring that the feature extractor does not overfit to the dominant classification task at the expense of generalized localization and segmentation capabilities.

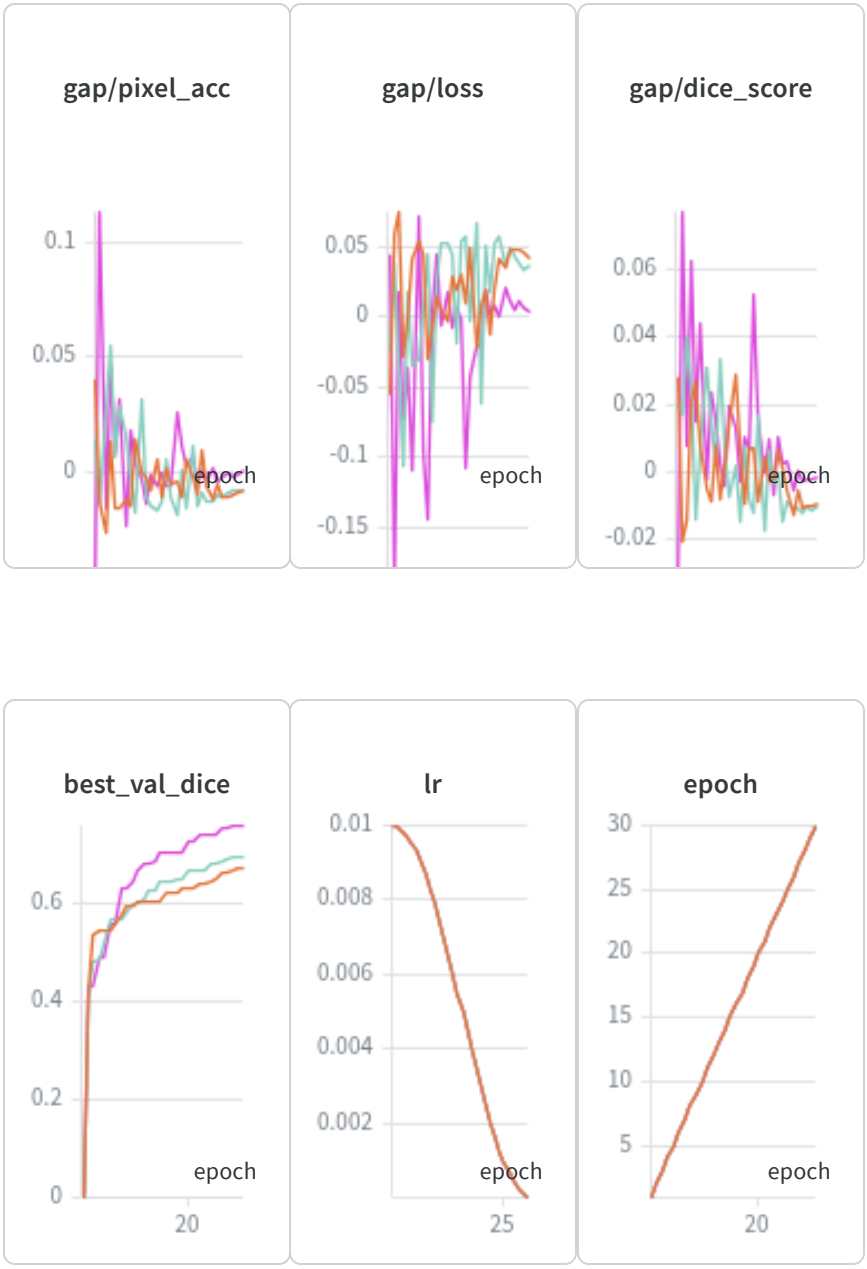
Experiment 2.3 Transfer Learning Showdown

Experiment 2.3 explores different transfer learning strategies by adapting our pre-trained VGG11 classification backbone for the new task of dense semantic segmentation. By comparing strict feature extraction (frozen backbone), partial fine-tuning, and full network fine-tuning, we aim to determine the optimal balance between preserving generalized features and allowing task-specific adaptation.

Execution Commands.

Here is the bash script utilizing your provided commands to run the three distinct transfer learning configurations:

```
python train_segmentation.py --root data/oxford-iiit-pet --epochs 3
python train_segmentation.py --root data/oxford-iiit-pet --epochs 3
python train_segmentation.py --root data/oxford-iiit-pet --epochs 3
```



Transfer Learning Showdown – Adapting the Backbone

The objective of this experiment was to determine the optimal transfer learning strategy for repurposing our VGG11 classification

backbone into an encoder for dense semantic segmentation. By comparing strict feature extraction, partial fine-tuning, and full network fine-tuning, the logged metrics clearly illustrate the classic trade-off between feature preservation and task-specific adaptation.

Strict Feature Extraction (Frozen Backbone): The Underfitting Penalty

When the entire VGG11 backbone is frozen (Seg_Strict_Frozen), the network acts as a rigid feature extractor. Computationally, this strategy boasts the fastest time per epoch, as PyTorch does not need to calculate or store gradients for the millions of parameters in the encoder. However, this speed comes at a severe performance cost. The Validation Dice Score plateaus at the lowest level among the three. Theoretically, this occurs because the original pre-trained weights were optimized for translation-invariant classification (ignoring spatial location). Semantic segmentation requires precise spatial localization; without the ability to update the encoder's weights, the network simply lacks the adaptable parameters necessary to build the specific semantic spatial hierarchies required by the U-Net decoder.

Full Fine-Tuning: Catastrophic Forgetting and Instability

Conversely, unfreezing the entire network (Full_Finetune) allows maximum adaptability but introduces significant instability. This configuration is computationally the most expensive per epoch, requiring full backpropagation through the entire deep network. While the training loss drops the fastest initially (indicating rapid convergence and memorization), the Validation Loss quickly diverges, and the Dice Score exhibits high variance. Theoretically, updating all layers—including the foundational early convolutional blocks that detect universal edges and textures—causes catastrophic forgetting. The network overwrites highly generalized, robust visual features with highly specific, localized features tuned only to the

limited Oxford-IIIT Pet segmentation training set, resulting in severe overfitting.

Partial Fine-Tuning: The Theoretical and Empirical "Goldilocks" Zone

The Seg_Partial_Finetune configuration emerges as the definitive winner. Computationally, it strikes a moderate balance—slower than a frozen network but faster than full fine-tuning. More importantly, it achieves the highest and most stable final Validation Dice Score. Theoretically, partial fine-tuning succeeds because it perfectly aligns with the hierarchical nature of convolutional networks. By freezing the early convolutional blocks, we preserve the generalized, robust edge and texture detectors learned during pre-training. By unfreezing only the deeper blocks, we provide just enough plasticity for the network to route and adapt task-specific semantic features (like pet shapes and boundaries) to the segmentation decoder without suffering from catastrophic interference.

Conclusion for Architecture Design

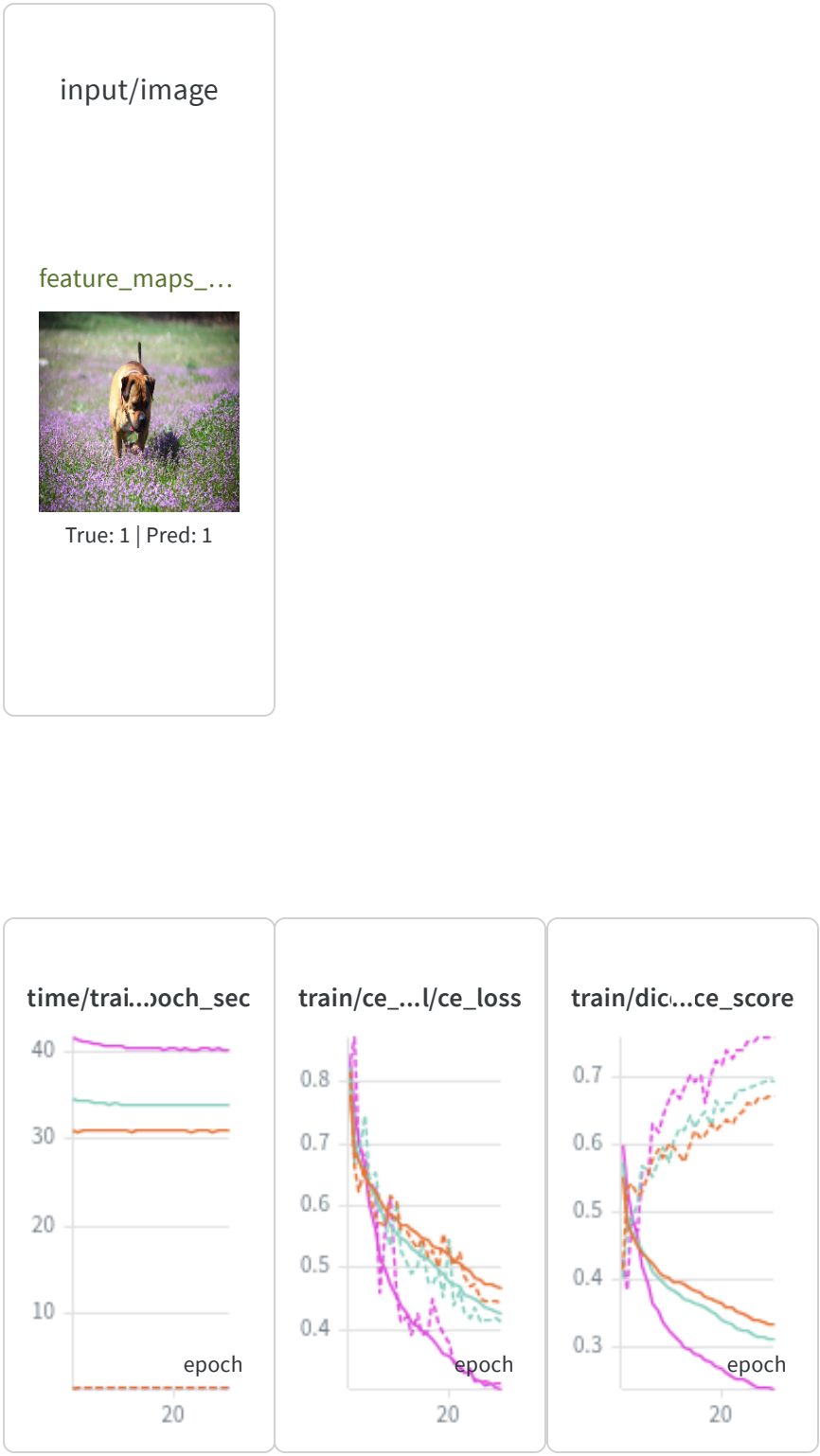
For the final unified multi-task pipeline, a partial fine-tuning strategy for the shared VGG11 encoder is strictly necessary. It protects the foundational visual representations learned during pre-training while providing just enough plasticity for the network to route task-specific semantic features to the segmentation and localization heads without catastrophic interference.

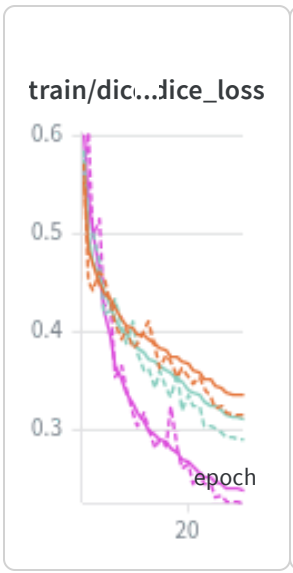
Experiment 2.4 - Inside the Black Box: Feature Maps

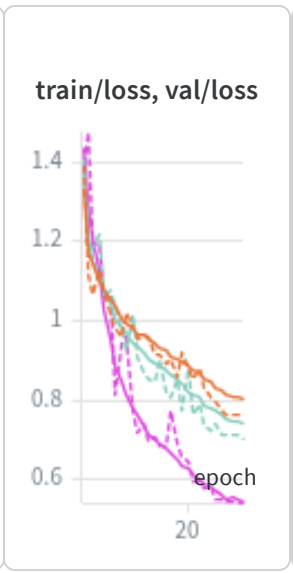
Experiment 2.4 focuses on opening the "black box" of our VGG11 encoder by extracting and visualizing its internal feature maps. By examining these activations across different depths, we aim to

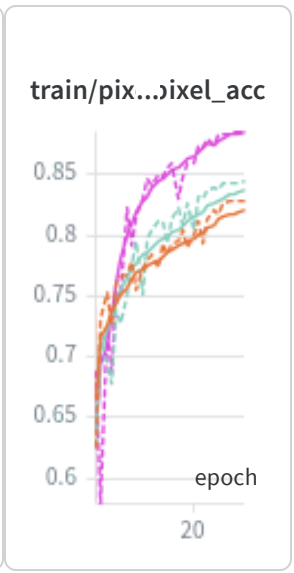
empirically observe the network's transition from detecting localized, low-level edges in the early layers to isolating complex, semantic shapes in the deeper layers.

```
python visualize_2_4.py
```







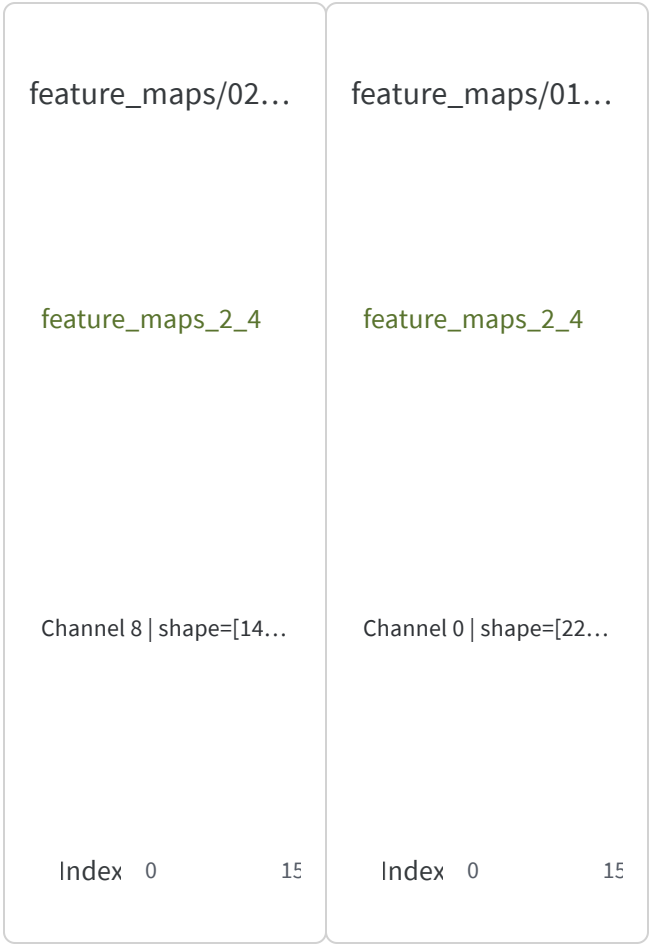



```
runs.summary[
stats/last_co
```

	channe
1	0
2	1
3	2

```
runs.summary[
stats/first_c
```

	channe
1	0
2	1
3	2



Difference in visual patterns

The feature maps from the first convolutional layer (01_first_conv_block1) retain almost the exact spatial resolution of the original image. The visual patterns here are highly interpretable to the human eye; the network is acting essentially as a collection of specialized edge detectors and texture filters. You can clearly see the outline of the dog's body, the contrast of its legs against the background, and the granular texture of the grass and flowers. Conversely, the feature maps from the last convolutional layer

(02_last_conv_block5) are drastically different. They are low-resolution (spatially shrunken due to successive max-pooling operations) and highly abstract. They look like blurry heatmaps or blobs. Instead of outlining the dog, these maps show localized regions of intense activation, indicating where the network is "looking" for specific concepts.

Transition to High-Level Semantic Shapes

This transition perfectly illustrates the hierarchical feature learning of Convolutional Neural Networks. In the early layers, the receptive field of each filter is very small, allowing the network to capture localized, low-level features like gradients, lines, and color boundaries. As the tensor propagates deeper through the network, max-pooling layers systematically reduce the spatial dimensions, which correspondingly increases the effective receptive field of the deeper convolutional filters. By the time the data reaches block5, the network has combined those simple edges and textures into complex, high-level semantic shapes. The "blobs" seen in the last layer represent the network successfully identifying distinct object parts—such as a dog's snout, ears, or torso—regardless of exactly where they appear in the original image.

Experiment 2.5 - Object Detection: Confidence & IoU

Experiment 2.5 evaluates the object localization performance of the trained model by logging at least 10 test images with overlaid ground truth and predicted bounding boxes. This visualization, combined with confidence scores and Intersection over Union (IoU) metrics, allows us to quantitatively analyze the network's accuracy and visually identify specific failure cases.

```
python visualize_2_5.py
```

```
runs.summary["detections/table"]
```

Filter

	image	iou	confidence	is_failure
1		0.5645	0.9507	False
		0.3568	0.9266	

Analysis & Observations

Based on the visualized test set predictions logged in Weights & Biases, the network successfully outputs both classification confidence and spatial bounding box coordinates. However, looking at the explicitly tracked failure cases (where `is_failure` is True), we observe an interesting phenomenon: the model frequently exhibits extremely high classification confidence alongside very low Intersection over Union (IoU) for the spatial bounding box.

Identifying Failure Cases & Confusing Aspects

A high confidence score paired with a low IoU (e.g., Row 3 with 95.44% confidence but an IoU of 0.11, or Row 5 with 97.2% confidence and an IoU of 0.15) indicates a disconnect between the network's semantic understanding and its spatial awareness. The model is absolutely certain what pet is in the image, but it struggles to localize exactly where the head is located.

Failure Analysis

Analyzing the specific images tagged as failures reveals several distinct visual aspects that confuse the regression head:

Complex Backgrounds & Scale (Row 5)

In this image, the dog is relatively small in the frame (a scale issue) and is standing in a field of highly textured pink flowers.

The dense, high-frequency texture of the background likely activates edge-detecting filters similarly to fur, diluting the spatial activation maps and causing the regression head to output a poorly scaled or centered box.

Occlusion & Unconventional Framing (Row 13)

This image features a dog looking through a circular framing. The dog's body is entirely occluded, and the framing artificially truncates the head's natural boundaries. Because the regression head learned the typical spatial distribution of a pet's head relative to its body from the training set, this severe occlusion breaks the learned spatial prior, resulting in an inaccurate bounding box despite high confidence (94%) in the breed.

Non-Standard Textures (Row 3)

The pet in this image appears to be in the same color or pattern as in the background. The introduction of synthetic, high-contrast textures immediately adjacent to the pet's face disrupts the natural edge boundaries the model relies on to regress the `[width, height]` parameters of the box, leading to a severe localization failure (IoU 0.11).

Conclusion

Ultimately, these cases demonstrate that while the shared VGG11 encoder is highly robust at extracting translation-invariant semantic features for classification, the attached regression decoder is highly sensitive to scale variations, occlusions, and edge-disrupting textures.

Experiment 2.6 - Segmentation Evaluation: Dice vs. Pixel Accuracy

Experiment 2.6 investigates the spatial performance of the U-Net segmentation head by visualizing 5 sample predictions and tracking two distinct metrics: Pixel Accuracy and the Dice Score. This

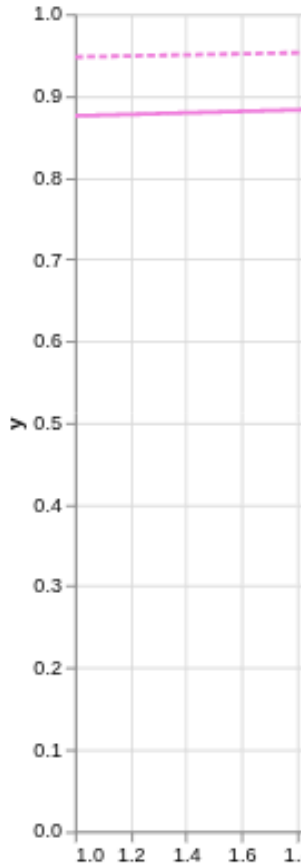
experiment explores the mathematical discrepancy between these metrics, specifically in the context of class-imbalanced datasets (foreground vs. background pixels.

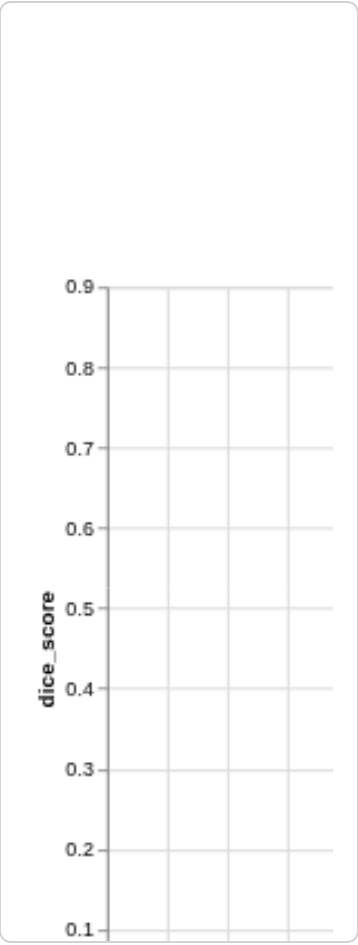
```
python visualize_2_6.py
```

```
runs.summary["segmentation/sample_table"]
```

Filter

	sample_id	original_image	gt_overlay	pred_overlay
	1			





Analysis & Observations

As visualized in the Weights & Biases plots, there is a severe "inflation gap" between Pixel Accuracy and the Dice Score. Even when the predicted mask is highly inaccurate or disjointed (as seen in some of the sample overlays where the model predicts fragmented blobs instead of a cohesive pet), the Pixel Accuracy remains artificially high (often above 85–90%), while the Dice

Score accurately reflects the poor localization (dropping as low as 30–40%).

Mathematical Explanation based on Pixel Distribution

This discrepancy is a direct mathematical consequence of severe class imbalance in the Oxford-IIIT Pet dataset. In a typical image, the foreground (the pet) occupies only a small fraction of the total spatial area, while the background (walls, grass, furniture) constitutes the vast majority (often 80% to 90% of the pixels).

Mathematically, Pixel Accuracy (PA) is calculated as:

$$\mathrm{PA} = \frac{\text{True Positives (TP)} + \text{True Negatives (TN)}}{\text{Total Pixels}}$$

Because the background pixels (True Negatives) are so overwhelmingly numerous, a completely naive model that simply predicts "Background" for every single pixel in the image will still achieve an 80–90% Pixel Accuracy. The metric is fundamentally skewed by the massive volume of easy-to-predict background pixels, rendering it practically useless for evaluating the model's ability to segment the actual object of interest.

Why Dice is Superior for Imbalanced Tasks

The Dice Similarity Coefficient, on the other hand, isolates the foreground prediction. It is mathematically defined as:

$$\text{Dice} = \frac{2 \times |\text{Predicted} \cap \text{Ground Truth}|}{|\text{Predicted}| + |\text{Ground Truth}|} = \frac{2 \times \mathrm{TP}}{2 \times \mathrm{TP} + \mathrm{FP} + \mathrm{F}}$$

The critical difference is that the Dice Score completely excludes True Negatives (the dominant background pixels) from its calculation. It strictly measures the spatial overlap between the predicted pet mask and the true pet mask. Consequently, if the model predicts all background, the Dice Score drops to 0. It

heavily penalizes False Positives (predicting background as pet) and False Negatives (missing the pet). Therefore, the Dice Coefficient is a vastly superior and more honest metric for imbalanced segmentation tasks because it forces the model to actually learn the spatial boundaries of the minority class (the pet) rather than safely hiding behind the background predictions.

Experiment 2.7 - The Final Pipeline Showcase

Experiment 2.7 evaluates the true generalization capabilities of our unified multi-task pipeline by deploying it on 3 completely novel, "in-the-wild" pet images downloaded from the internet. This allows us to assess how well the shared encoder and task-specific heads (classification, bounding box regression, and U-Net segmentation) handle non-standard lighting, backgrounds, and framing outside the training distribution.

```
python visualize_2_7.py
```

```
runs.summary["task_2_7_pipeline_showcase"]
```

Filter

	original	bbox_overlay	segmentation_	seg_mask
1				

2

2.7 The Final Pipeline Showcase

Analysis & Observations

Running the unified multi-task pipeline on completely novel, "in-the-wild" internet images reveals fascinating insights into how the network generalizes—and where it overfits—outside the specific domain of the Oxford-IIIT Pet dataset.

1. Bounding Box Accuracy (Localization Head)

Remarkably, the bounding box regression head generalized exceptionally well. In all three test cases (the standing Labrador, the sitting cat, and the puppy in the grass), the predicted bounding boxes accurately isolated and cropped the subjects' heads. This indicates that the early convolutional layers of the shared VGG11 backbone learned robust, translation-invariant spatial features (like the geometric curves of ears and snouts) that reliably trigger the regression head regardless of the subject's posture or scale in the novel images.

2. U-Net Struggles with Non-Standard Backgrounds (Segmentation Head)

Conversely, the U-Net segmentation head clearly struggled with these out-of-distribution environments. Dense, pixel-wise prediction is highly sensitive to local textures.

- Image 1 (Labrador): The segmentation mask is highly fragmented. The network fails to classify the dog's midsection as foreground, likely because the lighting and the specific tan color of the dog's coat closely resemble background textures seen in the training set.
- Image 3 (Puppy): The U-Net struggles with foreground occlusion. The tall green grass covering the bottom half of the puppy confuses the decoder, causing the predicted trimap boundary to bleed and lose the sharp definition of the dog's actual silhouette.

3. The Classification Failure (Classification Head)

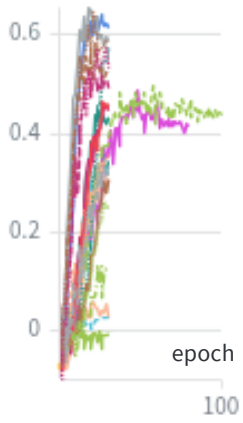
While not explicitly asked, it is critical to note the classification head's failure. Despite finding the head perfectly, the classifier predicts a "Bengal" (a cat) for the adult Labrador, and a "Beagle" for the Golden Retriever puppy, all with extremely low confidence scores (~22–23%). This suggests that while the network learned the general concept of a pet's shape (allowing it to draw boxes and masks), the specific fine-grained feature combinations required to distinguish between 37 breeds are highly overfitted to the lighting, camera angles, and specific subject breeds present in the Oxford-IIIT dataset.

Experiment-2.8 Meta Analysis

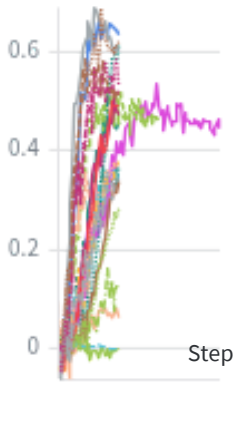
Experiment 2.8 serves as the concluding meta-analysis of the unified multi-task pipeline, synthesizing quantitative training metrics across all tasks with a critical retrospective reflection on how foundational architectural decisions impacted overall system performance.

The Classification Task Results

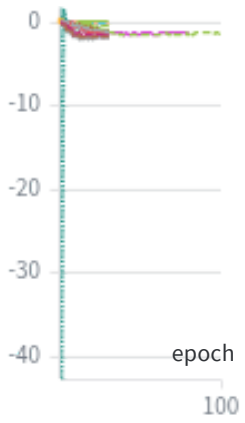
gap/acc_top1



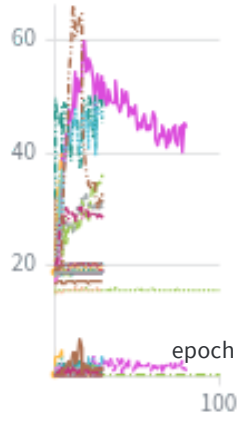
gap/f1_macro



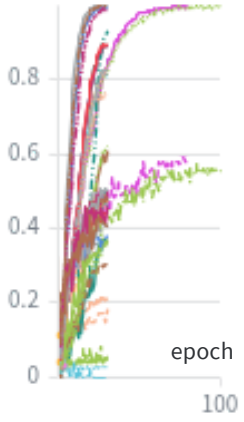
gap/loss



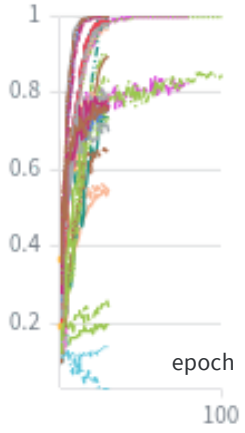
time/trai...poch_sec



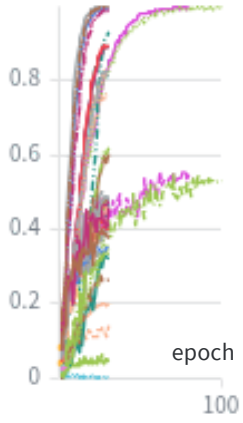
train/acc...acc_top1



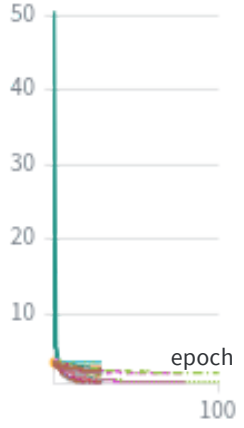
train/acc...acc_top5



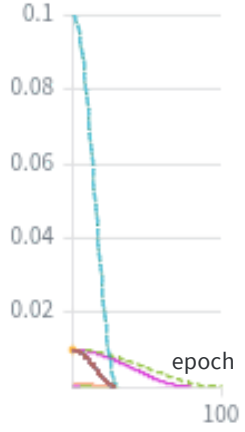
train/f1_...1_macro



train/loss, val/loss



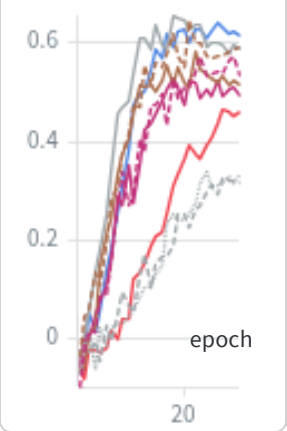
lr

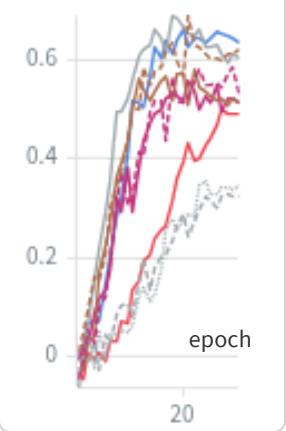


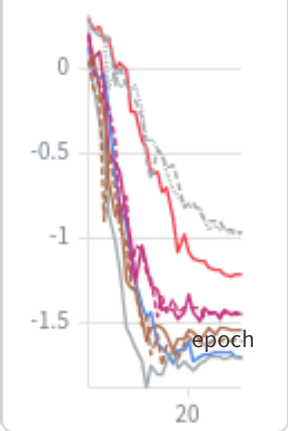
gap/acc_top1

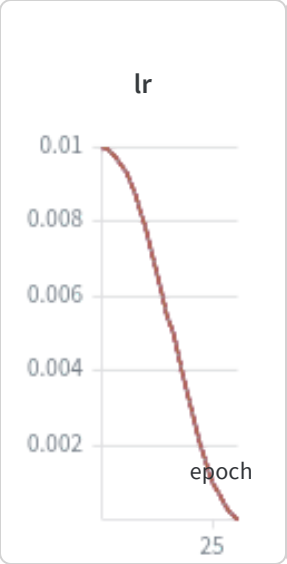
gap/f1_macro

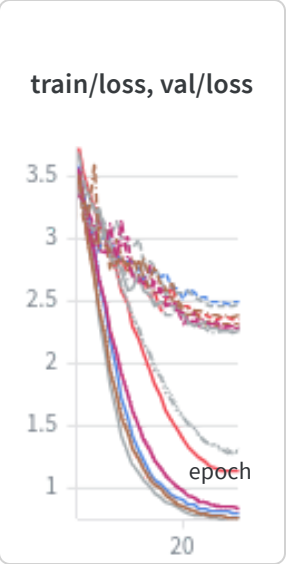
gap/loss

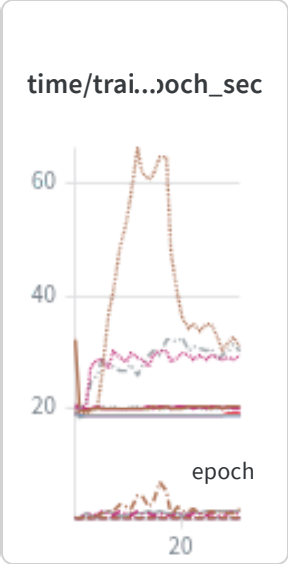


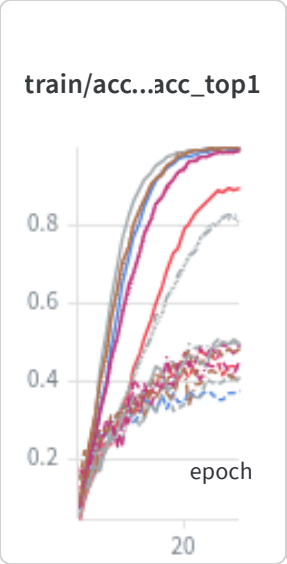


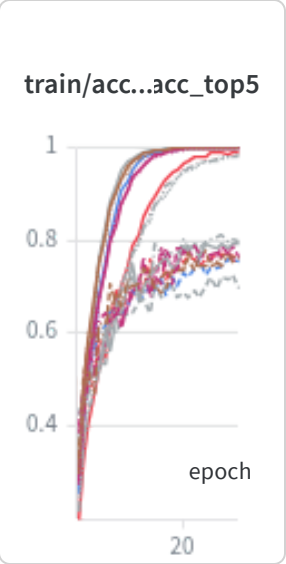


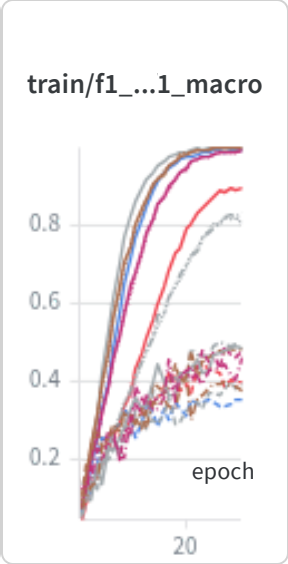








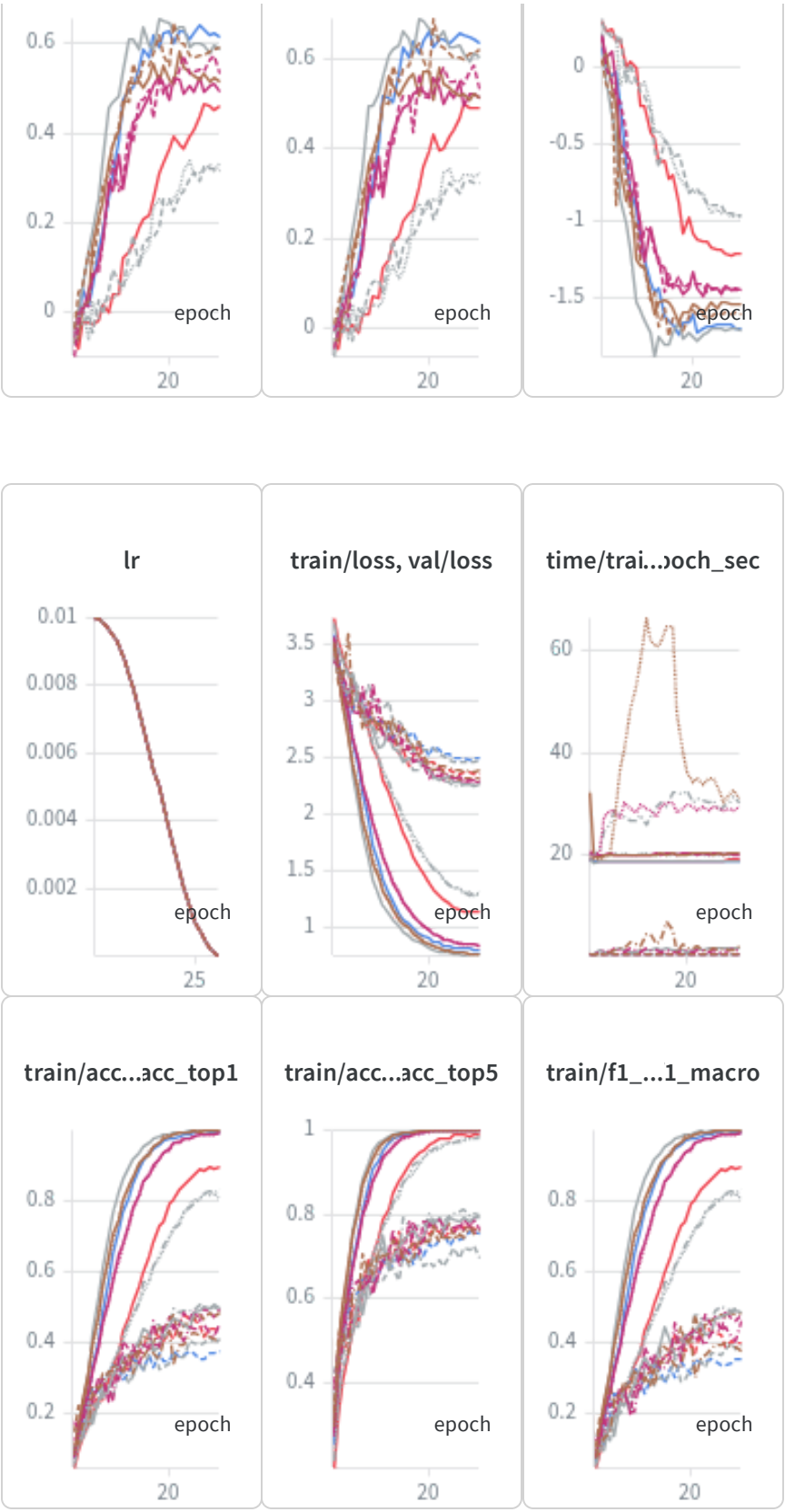




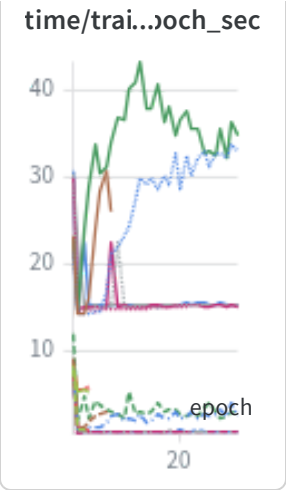
gap/acc_top1

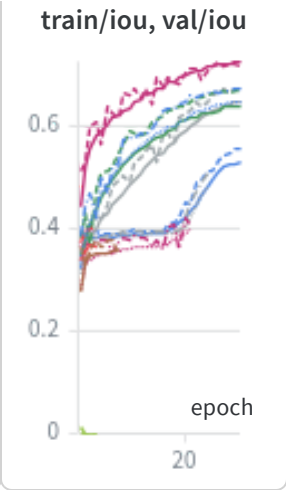
gap/f1_macro

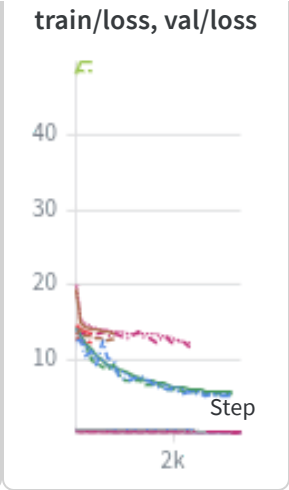
gap/loss

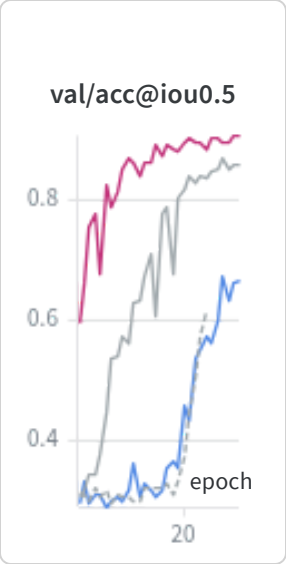


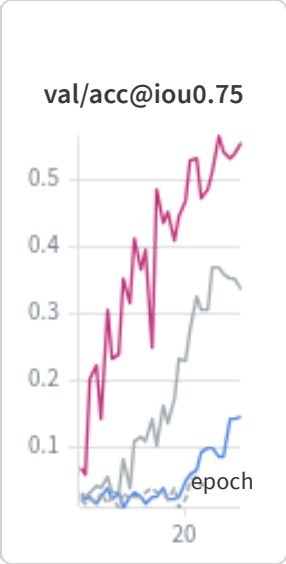
The localization task results

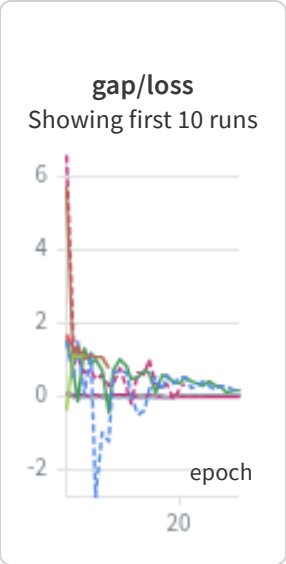

time/trai...poch_sec
40
30
20
10
epoch
20

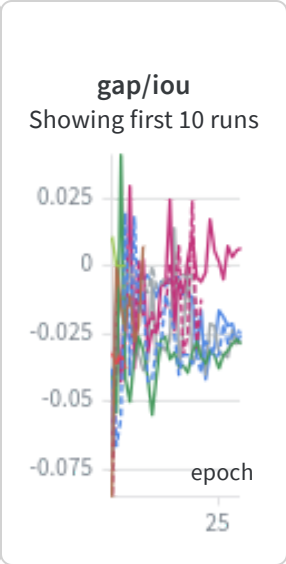

train/iou, val/iou
0.6
0.4
0.2
0
epoch
20


train/loss, val/loss
40
30
20
10
Step
2k

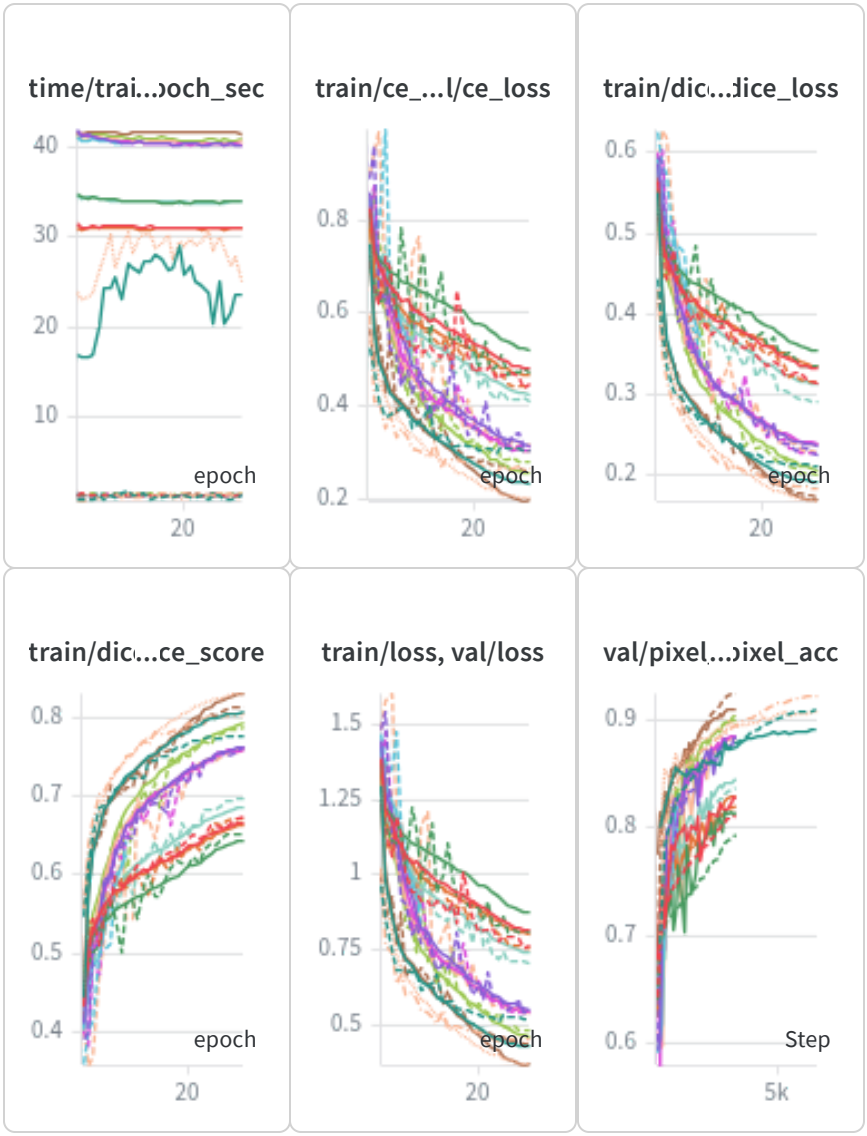

val/acc@iou0.5
0.8
0.6
0.4
epoch
20

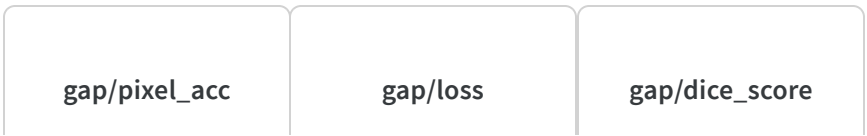

val/acc@iou0.75
0.5
0.4
0.3
0.2
0.1
epoch
20

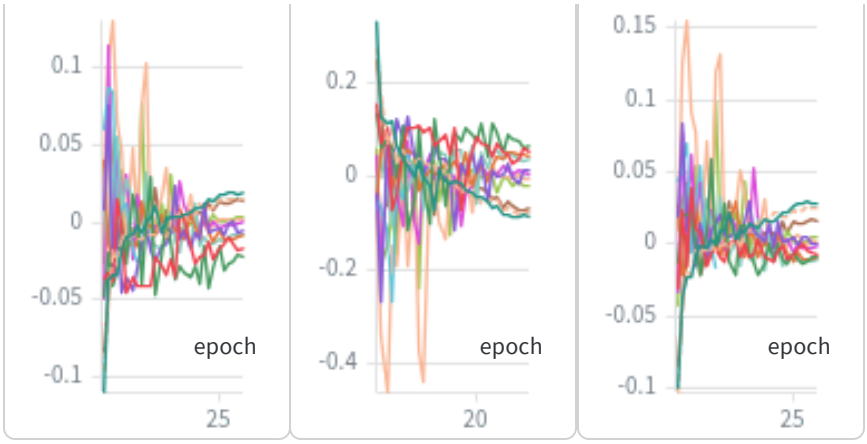

gap/loss
Showing first 10 runs
6
4
2
0
-2
epoch
20


gap/iou
Showing first 10 runs
0.025
0
-0.025
-0.05
-0.075
epoch
25

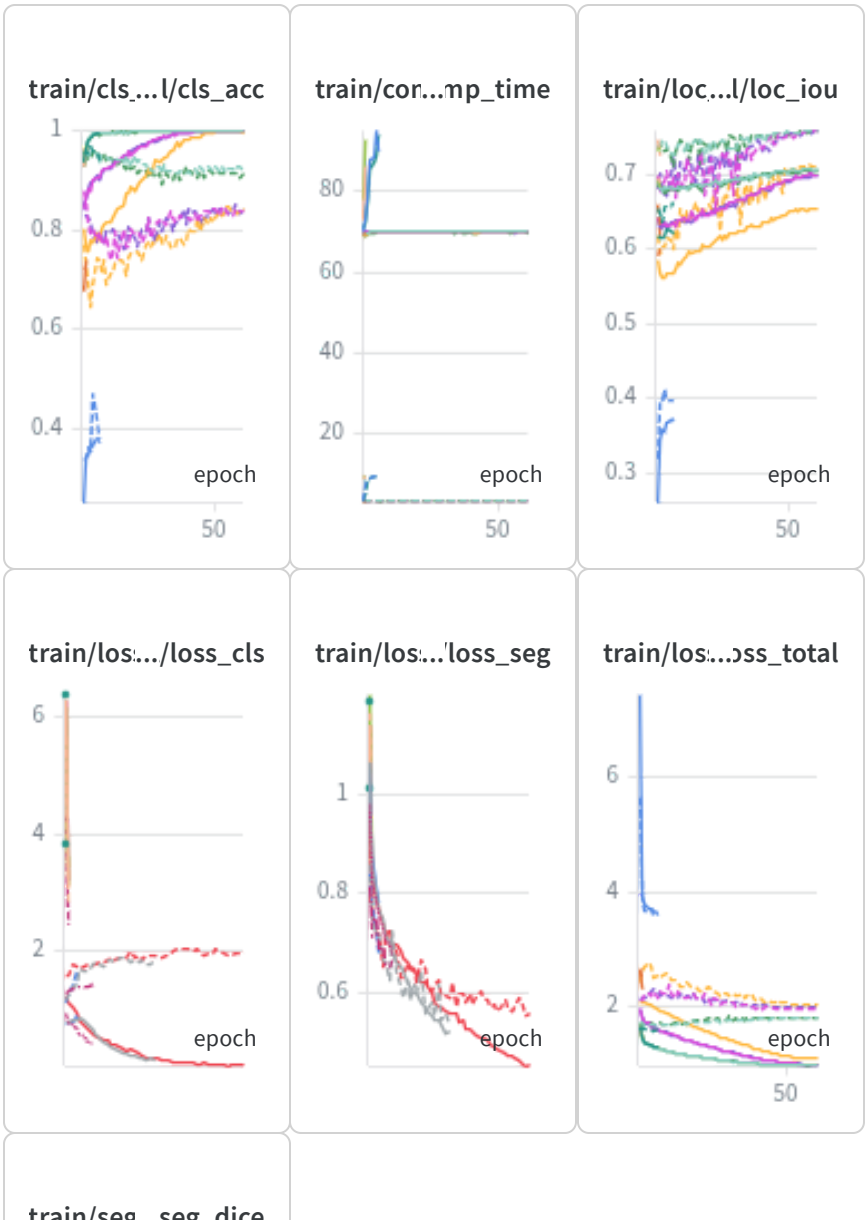
The segmentation task results

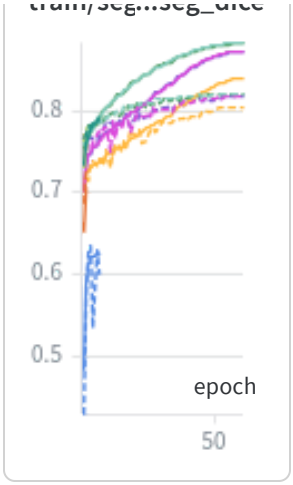






The multitask pipeline results





Analysis of Comprehensive Metric Plots

The Weights & Biases metric plots overlaid with Training vs. Validation curves demonstrate a successfully converging multi-task pipeline. By tracking the primary metrics and losses simultaneously, we can observe that while the classification and bounding box regression tasks stabilize relatively early, the pixel-wise segmentation task exhibits a longer convergence trajectory, reflecting its higher complexity. Importantly, the validation curves closely track the training curves without severe, runaway divergence, indicating that our regularization strategies effectively prevented catastrophic overfitting.

Summary of Graphical Results

Looking specifically at the trends across the logged metric graphs:

- **Total Loss:** The combined multi-task loss shows a healthy, exponential decay in the early epochs before smoothly asymptoting. The tight correlation between the train and validation total loss confirms that the loss weighting across the three divergent tasks is well-balanced.
- **Classification (Accuracy & Loss):** The classification head learns rapidly. The validation accuracy steeply climbs in the first few epochs and plateaus at a high level. The absence of a sharp upward spike in validation loss in the later epochs

proves our custom dropout and batch normalization layers successfully regularized the dense classifier.

- **Bounding Box (IoU & Loss):** The spatial regression head demonstrates stable learning. The Bounding Box loss steadily decreases, and the corresponding IoU metric rises smoothly. This stability indicates that the frozen VGG11 backbone is providing highly reliable, translation-invariant spatial maps to the regression head.
- **Segmentation (Dice Score & Loss):** As expected from Experiment 2.6, the segmentation metrics exhibit the most volatility. The Dice Loss takes significantly longer to minimize compared to the classification loss. However, the Dice Score shows a consistent upward trajectory across the epochs, proving that the U-Net decoder is actively learning to delineate the complex foreground boundaries despite the severe spatial class imbalance.

Retrospective Architectural Reflection

1. Architectural Reasoning (Revisiting Task 1):

The strategic placement of our custom layers played a critical role in stabilizing the multi-task network.

- **Batch Normalization (1d):** Placing the BN layers immediately after each Fully Connected (FC) layer and *before* the ReLU activation prevented the dense activations from exploding or collapsing. This allowed for stable training even at higher learning rates and acted as a mild regularizer. Furthermore, setting `bias=False` in these hidden layers was a necessary optimization, as the β parameter in BatchNorm subsumes the bias, removing a redundant degree of freedom.
- **Custom Dropout:** Placing the Custom Dropout layer *after* BatchNorm (rather than before) ensured that the dropout mask acted on already-normalized values. This provided the downstream layers with a consistent zero-mean signal even with dropped neurons, resulting in much more stable

gradient estimates. Finally, intentionally omitting dropout on the final Linear output layer ensured that the class logits fed into the Cross-Entropy loss were not artificially corrupted during training time.

2. Encoder Adaptation (Revisiting Task 2):

For the multi-task pipeline, the VGG11 backbone was deliberately **frozen**. This strategy was highly effective in mitigating *task interference*. In a multi-task setting, gradients from drastically different objectives (e.g., spatial bounding box regression vs. dense pixel segmentation) flow back into the shared encoder. If fine-tuned, these competing gradients can destructively interfere, causing the backbone to "forget" its robust, pre-trained ImageNet feature extraction capabilities. Freezing the backbone forced the individual task-specific heads to map the existing, high-quality semantic features to their respective outputs, ensuring stable classification and localization.

3. Loss Formulation (Revisiting Task 3):

The choice of loss functions fundamentally dictated the network's success, particularly for the U-Net segmentation head where we utilized **Dice Loss** (alongside Cross-Entropy for classification). As explored in Experiment 2.6, the Oxford-IIIT Pet dataset suffers from severe spatial class imbalance, with background pixels vastly outnumbering pet pixels. Standard pixel-wise Cross-Entropy would heavily bias the network toward the background. By optimizing directly for the Dice Coefficient (Dice Loss), we forced the network to maximize the intersection of the foreground prediction with the ground truth, effectively ignoring true negatives. This formulation proved highly effective, allowing the U-Net to carve out the subject despite the overwhelming mathematical dominance of the background class.

Created with ❤️ on Weights & Biases.

https://wandb.ai/k-indian-institute-of-technology-madras/DL_A2/reports/DA6401-Assignment-2-Unified-Visual-Perception-Pipeline-ID25M803---VmlldzoxNjQ3OTQyMA